НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»
Фізико-технічний інститут

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №3

з дисципліни «Криптографія»

Тема: «Криптоаналіз афінної біграмної підстановки»

Виконали:
студенти групи ФБ-43
Оліневич Богдан
та Візгін Климент
Варіант 7

1. Мета комп’ютерного практикуму

Навчитися на практиці ламати шифри частотним методом — на прикладі афінної біграмної підстановки (моноалфавітної заміни біграм). Заодно потренуватися в модулярній арифметиці: рахувати обернені елементи розширеним алгоритмом Евкліда, розв’язувати лінійні порівняння з одним чи кількома розв’язками, а ще придумати критерії, за якими програма сама відрізнятиме осмислений текст від сміття.

2. Постановка задачі та варіант завдання

Індивідуальний варіант: наш варіант №7 (шифртекст лежить у файлі 07.txt).

Нам дали шифртекст із файлу 07.txt: 7102 символи, тобто 3551 неперетинна біграма. Його отримали з російського відкритого тексту афінною біграмною підстановкою. У тексті немає пробілів, розділових знаків і великих літер, «ё» замінено на «е», а «ъ» — на «ь». Тож в алфавіті m = 31 літера (номери від 0 до 30): «абвгдежзийклмнопрстуфхцчшщьыэюя», де «ь» стоїть на місці вилученого «ъ» (позиція 26), а «ы» має позицію 27.

Кожній неперетинній біграмі відкритого тексту (x_{2i-1}, x_{2i}) відповідає число X_i = x_{2i-1} * m + x_{2i} від 0 до m^2 - 1 (де m^2 = 31^2 = 961). Кожну біграму X_i шифрують окремо за таким правилом:

Формула шифрування: Y_i = (a * X_i + b) mod 961, де 0 < a < 961, gcd(a, 31) = 1, 0 <= b < 961.

Формула розшифрування: X_i = a^(-1) * (Y_i - b) mod 961, де a^(-1) — обернений елемент до a за модулем 961.

Що потрібно було написати на Python:

- **1. Математичний модуль:** функції для НСД і коефіцієнтів Безу (розширений алгоритм Евкліда), оберненого елемента за модулем та розв'язування порівнянь a * x ≡ b (mod n) — так, щоб усе працювало і тоді, коли розв'язків кілька.
- **2. Частотний аналіз шифртексту:** взяти функції підрахунку частот біграм та ентропії з КП №1 і знайти 5 найчастіших неперетинних біграм шифртексту варіанта №7.
- **3. Генерація кандидатів на ключ:** перебрати співставлення пар із 5 найчастіших біграм російської мови («ст», «но», «то», «на», «ен») та 5 найчастіших біграм шифртексту й для кожного розв'язати порівняння (Y* - Y**) ≡ a * (X* - X**) (mod 961).
- **4. Автоматичний розпізнавач змістовного тексту:** розшифрувати текст кожним кандидатом (a, b) і програмно відібрати той варіант, що схожий на справжній російський текст, за статистичними критеріями.

3. Хід роботи

Етап 1. Реалізація математичних операцій модулярної арифметики

```python
import math
import re
from collections import Counter


# ==================== ПОЧАТОК ЕТАПУ 1 ====================
print("=" * 60)
print("ПОЧАТОК ЕТАПУ 1: МАТЕМАТИЧНІ ОПЕРАЦІЇ МОДУЛЯРНОЇ АРИФМЕТИКИ")
print("=" * 60)

def extended_gcd(a: int, b: int):
    r0, r1 = a, b
    u0, u1 = 1, 0
    v0, v1 = 0, 1
    while r1 != 0:
        q = r0 // r1
        r0, r1 = r1, r0 - q * r1
        u0, u1 = u1, u0 - q * u1
        v0, v1 = v1, v0 - q * v1
    return r0, u0, v0

def mod_inverse(a: int, n: int):
    a %= n
    d, u, _ = extended_gcd(a, n)
    if d != 1:
        return None
    return u % n

def solve_linear_congruence(a: int, b: int, n: int):
    a %= n
    b %= n
    d, _, _ = extended_gcd(a, n)
    if d == 1:
        inv = mod_inverse(a, n)
        return [(b * inv) % n]
    if b % d != 0:
        return []
    a1 = a // d
    b1 = b // d
    n1 = n // d
    inv = mod_inverse(a1, n1)
    x0 = (b1 * inv) % n1
    return [(x0 + k * n1) % n for k in range(d)]

test_inv = mod_inverse(200, 961)
test_single = solve_linear_congruence(128, 614, 961)
test_multi = solve_linear_congruence(155, 248, 961)
print(f"Перевірка оберненого елемента: 200^(-1) mod 961 = {test_inv}")
print(f"Порівняння з 1 розв'язком (128*a = 614 mod 961): {test_single}")
print(f"Порівняння з декількома розв'язками (155*a = 248 mod 961), кількість = {len(test_multi)}:")
print(f"  {test_multi}")
```

```
============================================================
ПОЧАТОК ЕТАПУ 1: МАТЕМАТИЧНІ ОПЕРАЦІЇ МОДУЛЯРНОЇ АРИФМЕТИКИ
============================================================
Перевірка оберненого елемента: 200^(-1) mod 961 = 764
Порівняння з 1 розв'язком (128*a = 614 mod 961): [200]
Порівняння з декількома розв'язками (155*a = 248 mod 961), кількість = 31:
  [14, 45, 76, 107, 138, 169, 200, 231, 262, 293, 324, 355, 386, 417, 448, 479, 510, 541, 572, 603, 634, 665, 696, 727, 758, 789, 820, 851, 882, 913, 944]
------------------------------------------------------------
КІНЕЦЬ ЕТАПУ 1
============================================================
```

Що на скріншотах (Етап 1): тут три базові функції, на яких тримається весь криптоаналіз у кільці лишків за модулем m^2 = 961:

Функція extended_gcd(a, b): це розширений алгоритм Евкліда. У циклі while r1 != 0 щоразу рахується неповна частка q = r0 // r1 і оновлюються остачі (r0, r1) та послідовності коефіцієнтів (u0, u1) і (v0, v1) за формулою u_{i+1} = u_{i-1} - q_i * u_i. Коли цикл закінчується, функція віддає трійку (d, u, v), для якої виконується тотожність Безу: a * u + b * v = d = gcd(a, b).

Функція mod_inverse(a, n): шукає обернений елемент a^(-1) mod n. Вона викликає extended_gcd(a % n, n). Якщо d != 1, оберненого елемента просто немає, тож повертається None. Якщо ж d == 1, то з рівності a * u + n * v = 1 видно, що a * u ≡ 1 (mod n), і відповідь — u % n.

Функція solve_linear_congruence(a, b, n): розв'язує порівняння a * x ≡ b (mod n). Спершу рахуємо d = gcd(a, n). Якщо d == 1, розв'язок один: x = (b * a^(-1)) mod n. Якщо d > 1 і b % d != 0, розв'язків немає, і функція повертає порожній список []. А якщо d > 1 і b ділиться на d, скорочуємо a, b та n на d (a1 = a // d, b1 = b // d, n1 = n // d), знаходимо базовий розв'язок x0 = (b1 * a1^(-1)) mod n1 і збираємо всі d розв'язків вигляду (x0 + k * n1) mod n для k = 0, 1, ..., d - 1.

З консольного виводу видно, що тести пройшли: обернений елемент до 200 за модулем 961 дорівнює 764 (бо 200 * 764 = 152800 = 159 * 961 + 1 ≡ 1 (mod 961)). Порівняння 128 * a ≡ 614 (mod 961), де gcd(128, 961) = 1, має єдиний розв'язок [200]. А в порівнянні 155 * a ≡ 248 (mod 961) gcd(155, 961) = 31, і 248 ділиться на 31 (155 = 5 * 31, 248 = 8 * 31), тому функція чесно повертає всі 31 розв'язок із кроком n1 = 31: [14, 45, 76, 107, 138, 169, 200, ..., 944].

Етап 2. Частотний аналіз біграм шифртексту (за допомогою коду з КП №1)

```python
print(f"  {test_multi}")

print("-" * 60)
print("КІНЕЦЬ ЕТАПУ 1")
print("=" * 60 + "\n")
# ==================== КІНЕЦЬ ЕТАПУ 1 ====================


# ==================== ПОЧАТОК ЕТАПУ 2 ====================
print("=" * 60)
print("ПОЧАТОК ЕТАПУ 2: ЧАСТОТНИЙ АНАЛІЗ БІГРАМ ШИФРТЕКСТУ (ЗА КОДОМ КП №1)")
print("=" * 60)

RUSSIAN_LETTERS = "абвгдежзийклмнопрстуфхцчшщьыэюя"
ALPHABET_WITH_SPACE = set(RUSSIAN_LETTERS + " ")
M = len(RUSSIAN_LETTERS)
M2 = M * M
CHAR_TO_IDX = {ch: idx for idx, ch in enumerate(RUSSIAN_LETTERS)}
IDX_TO_CHAR = {idx: ch for idx, ch in enumerate(RUSSIAN_LETTERS)}

def preprocess(raw_text: str, keep_spaces: bool = False) -> str:
    text = raw_text.lower()
    text = text.replace("ё", "е").replace("ъ", "ь")
    text = "".join(ch if ch in ALPHABET_WITH_SPACE else " " for ch in text)
    text = re.sub(r" +", " ", text).strip()
    if not keep_spaces:
        text = text.replace(" ", "")
    return text

def calc_entropy(counts: Counter, total: int) -> float:
    h = 0.0
    for c in counts.values():
        p = c / total
        h -= p * math.log2(p)
    return h

def bigram_to_num(bg: str) -> int:
    return CHAR_TO_IDX[bg[0]] * M + CHAR_TO_IDX[bg[1]]

def num_to_bigram(num: int) -> str:
    return IDX_TO_CHAR[num // M] + IDX_TO_CHAR[num % M]

def analyze_text(text: str, label: str = "ШИФРТЕКСТ БЕЗ ПРОБІЛІВ"):
    total = len(text)
    counts = Counter(text)
    h1 = calc_entropy(counts, total)

    bg_over = [text[i:i+2] for i in range(total - 1)]
    h2_over = calc_entropy(Counter(bg_over), len(bg_over)) / 2.0

    bg_nonover = [text[i:i+2] for i in range(0, total - 1, 2)]
```

```
    bg_nonover = [text[i:i+2] for i in range(0, total - 1, 2)]
    h2_nonover = calc_entropy(Counter(bg_nonover), len(bg_nonover)) / 2.0

    print(f"{label} (алфавіт із {M} символів, довжина: {total})")
    print("Топ-5 символів (символ, кількість, частка):")
    for ch, n in counts.most_common(5):
        display_ch = f"'{ch}'"
        print(f"{display_ch:>15} {n:>10} {n/total:>8.4f}")

    print("Топ-5 біграм, що ПЕРЕТИНАЮТЬСЯ:")
    for bg, n in Counter(bg_over).most_common(5):
        display_bg = f"'{bg}' (Y={bigram_to_num(bg):>3})"
        print(f"{display_bg:>15} {n:>10} {n/len(bg_over):>8.4f}")

    print("Топ-5 біграм, що НЕ перетинаються:")
    top5_nonover = Counter(bg_nonover).most_common(5)
    for bg, n in top5_nonover:
        display_bg = f"'{bg}' (Y={bigram_to_num(bg):>3})"
        print(f"{display_bg:>15} {n:>10} {n/len(bg_nonover):>8.4f}")

    print(f"{'H1':<15} = {h1:.4f} біт/симв.")
    print(f"{'H2 (перетин.)':<15} = {h2_over:.4f} біт/симв.")
    print(f"{'H2 (без перет.)':<15} = {h2_nonover:.4f} біт/симв.")
    return [bg for bg, _ in top5_nonover]

file_name = "07.txt"
with open(file_name, encoding="cp1251", errors="ignore") as f:
    raw_data = f.read()

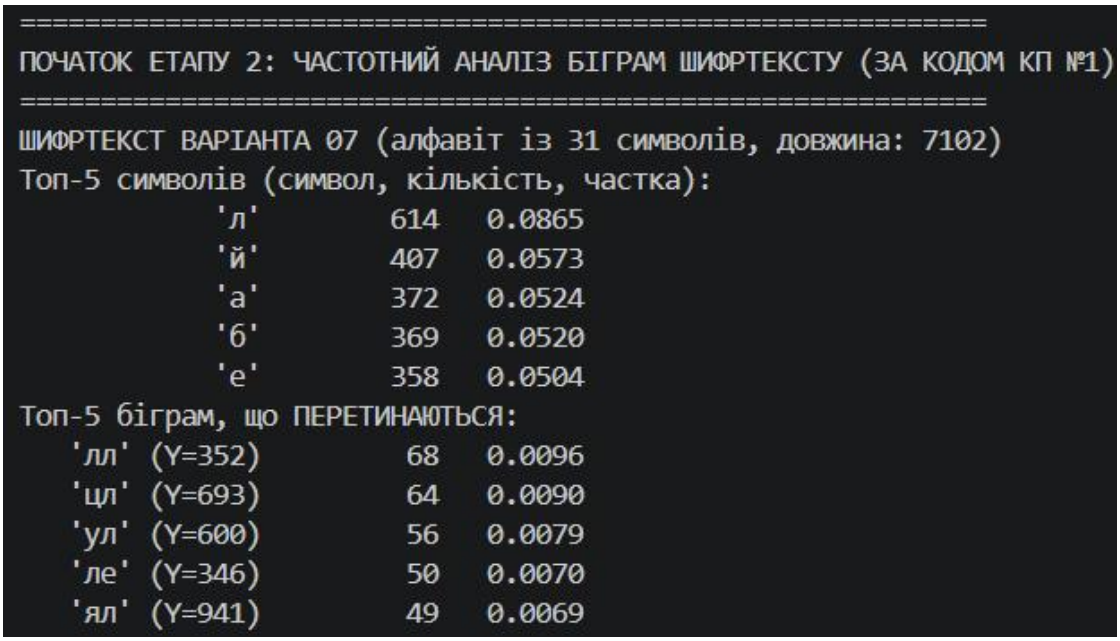
cipher_text = preprocess(raw_data, keep_spaces=False)
cipher_top5_bigrams = analyze_text(cipher_text, "ШИФРТЕКСТ ВАРІАНТА 07")

print("-" * 60)
print("КІНЕЦЬ ЕТАПУ 2")
print("=" * 60 + "\n")
# ==================== КІНЕЦЬ ЕТАПУ 2 ====================

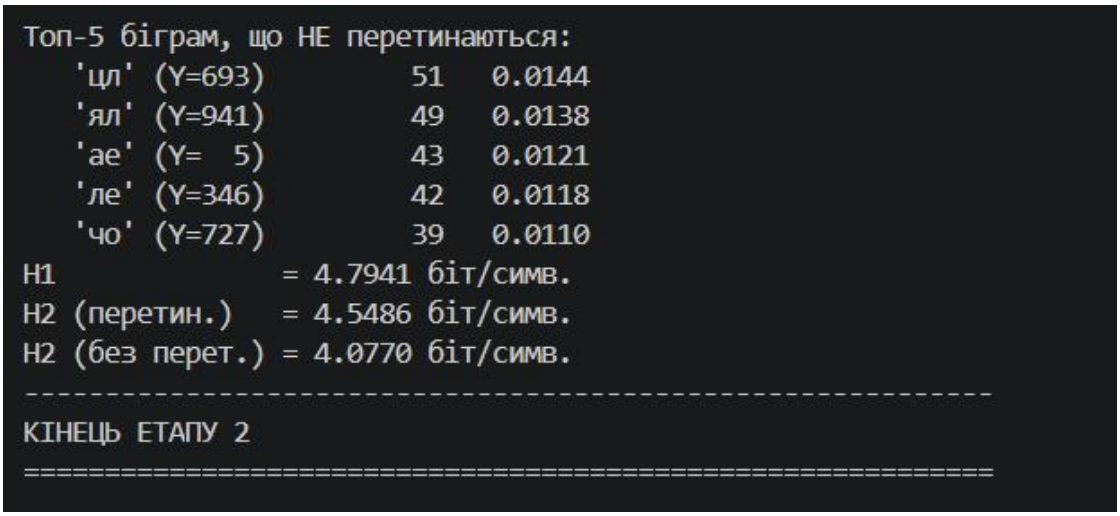

# ==================== ПОЧАТОК ЕТАПУ 3 ====================
print("=" * 60)
print("ПОЧАТОК ЕТАПУ 3: ПЕРЕБІР СПІВСТАВЛЕНЬ БІГРАМ ТА ПОШУК КАНДИДАТІВ НА КЛЮЧ (a, b)")
print("=" * 60)

LANG_TOP5_BIGRAMS = ["ст", "но", "то", "на", "ен"]

def generate_key_candidates(lang_bigrams, cipher_bigrams):
    x_nums = [bigram_to_num(bg) for bg in lang_bigrams]
    y_nums = [bigram_to_num(bg) for bg in cipher_bigrams]
    candidates = set()
```



```
============================================================
ПОЧАТОК ЕТАПУ 2: ЧАСТОТНИЙ АНАЛІЗ БІГРАМ ШИФРТЕКСТУ (ЗА КОДОМ КП №1)
============================================================
ШИФРТЕКСТ ВАРІАНТА 07 (алфавіт із 31 символів, довжина: 7102)
Топ-5 символів (символ, кількість, частка):
            'л'        614   0.0865
            'й'        407   0.0573
            'а'        372   0.0524
            'б'        369   0.0520
            'е'        358   0.0504
Топ-5 біграм, що ПЕРЕТИНАЮТЬСЯ:
   'лл' (Y=352)         68   0.0096
   'цл' (Y=693)         64   0.0090
   'ул' (Y=600)         56   0.0079
   'ле' (Y=346)         50   0.0070
   'ял' (Y=941)         49   0.0069
```



```
Топ-5 біграм, що НЕ перетинаються:
   'цл' (Y=693)         51   0.0144
   'ял' (Y=941)         49   0.0138
   'ае' (Y=  5)         43   0.0121
   'ле' (Y=346)         42   0.0118
   'чо' (Y=727)         39   0.0110
H1              = 4.7941 біт/симв.
H2 (перетин.)   = 4.5486 біт/симв.
H2 (без перет.) = 4.0770 біт/симв.
------------------------------------------------------------
КІНЕЦЬ ЕТАПУ 2
============================================================
```

Що на скріншотах (Етап 2): тут код нормалізації тексту та статистичного аналізу. Ми взяли його з першої лабораторної й підігнали під 31-літерний алфавіт:

Константа RUSSIAN_LETTERS та кодування біграм: рядок «абвгдежзийклмнопрстуфхцчшщьыэюя» задає відповідність між літерами та індексами 0..30 (M = 31, M2 = 961). Функція bigram_to_num(bg) рахує числове значення біграми Y = y_1 * 31 + y_2, а num_to_bigram(num) робить зворотне перетворення через цілу частину num // 31 та остачу num % 31.

Функції preprocess, calc_entropy та analyze_text: очищають шифртекст 07.txt (читаємо його в cp1251) від переносів рядків, будують списки перетинних біграм bg_over (крок 1) та неперетинних bg_nonover (крок 2 — саме так шифр ділить текст), а ще рахують ентропію окремих літер H1 і питому ентропію на символ для біграм H2 = H(біграм) / 2.

У консолі бачимо, що очищений шифртекст має 7102 символи (3551 неперетинна біграма). Для Етапу 3 нам потрібні саме неперетинні біграми, бо шифр ріже текст на блоки по 2 символи. Топ-5 такі: «цл» (Y = 693, 51 раз, 1.44%), «ял» (Y = 941, 49 разів, 1.38%), «ае» (Y = 5, 43 рази, 1.21%), «ле» (Y = 346, 42 рази, 1.18%) та «чо» (Y = 727, 39 разів, 1.10%). Ентропія окремих літер шифртексту висока: H1 = 4.7941 біт/символ, майже як максимум log2(31) ≈ 4.9542. А от ентропія неперетинних біграм H2 (без перет.) = 4.0770 біт/символ, бо їхній частотний профіль повністю зберігся.

Етап 3. Перебір співставлень частих біграм та пошук кандидатів на ключ (a, b)

```
    total_checked = 0
    multi_solution_cases = 0
    no_solution_cases = 0

    for i in range(len(x_nums)):
        for j in range(len(x_nums)):
            if i == j:
                continue
            for k in range(len(y_nums)):
                for l in range(len(y_nums)):
                    if k == l:
                        continue
                    total_checked += 1
                    dx = (x_nums[i] - x_nums[j]) % M2
                    dy = (y_nums[k] - y_nums[l]) % M2
                    a_solutions = solve_linear_congruence(dx, dy, M2)
                    if len(a_solutions) == 0:
                        no_solution_cases += 1
                    elif len(a_solutions) > 1:
                        multi_solution_cases += 1
                    for a in a_solutions:
                        d_a, _, _ = extended_gcd(a, M)
                        if a > 0 and d_a == 1:
                            b = (y_nums[k] - a * x_nums[i]) % M2
                            candidates.add((a, b))

    print(f"Топ-5 біграм російської мови: {[f'{bg} (X={bigram_to_num(bg)})' for bg in lang_bigrams]}")
    print(f"Топ-5 біграм шифртексту:      {[f'{bg} (Y={bigram_to_num(bg)})' for bg in cipher_bigrams]}")
    print(f"Усього розглянуто пар співставлень: {total_checked}")
    print(f"Випадків без розв'язків лінійного порівняння: {no_solution_cases}")
    print(f"Випадків із декількома розв'язками (по 31 розв'язку): {multi_solution_cases}")
    print(f"Знайдено унікальних допустимих кандидатів на ключ (a, b): {len(candidates)}")
    return sorted(candidates)

key_candidates = generate_key_candidates(LANG_TOP5_BIGRAMS, cipher_top5_bigrams)
print("Перші 10 кандидатів на ключ (a, b):", key_candidates[:10])

print("-" * 60)
print("КІНЕЦЬ ЕТАПУ 3")
print("=" * 60 + "\n")
# ==================== КІНЕЦЬ ЕТАПУ 3 ====================
```

```
ПОЧАТОК ЕТАПУ 3: ПЕРЕБІР СПІВСТАВЛЕНЬ БІГРАМ ТА ПОШУК КАНДИДАТІВ НА КЛЮЧ (a, b)
============================================================
Топ-5 біграм російської мови: ['ст (X=545)', 'но (X=417)', 'то (X=572)', 'на (X=403)', 'ен (X=168)']
Топ-5 біграм шифртексту:      ['цл (Y=693)', 'ял (Y=941)', 'ае (Y=5)', 'ле (Y=346)', 'чо (Y=727)']
Усього розглянуто пар співставлень: 400
Випадків без розв'язків лінійного порівняння: 32
Випадків із декількома розв'язками (по 31 розв'язку): 8
Знайдено унікальних допустимих кандидатів на ключ (a, b): 260
Перші 10 кандидатів на ключ (a, b): [(3, 437), (4, 600), (5, 848), (14, 621), (17, 579), (22, 875), (25, 842), (27, 278), (35, 166), (41, 780)]
------------------------------------------------------------
КІНЕЦЬ ЕТАПУ 3
============================================================
```

Що на скріншотах (Етап 3): тут код функції generate_key_candidates. У ній чотири вкладені цикли: два перебирають індекси (i, j) двох різних біграм відкритого тексту (X*, X**), ще два — індекси (k, l) двох різних біграм шифртексту (Y*, Y**). З 5 елементів виходить 5 *

4 = 20 впорядкованих пар, тож усього перевіряємо 20 * 20 = 400 комбінацій.

Для кожної четвірки біграм віднімаємо одне рівняння системи від іншого: Y* ≡ a * X* + b (mod 961) та Y** ≡ a * X** + b (mod 961). Так b зникає, і лишається порівняння тільки для a: dx * a ≡ dy (mod 961), де dx = (X* - X**) mod 961, а dy = (Y* - Y**) mod 961. Кожен розв'язок a, який видала solve_linear_congruence(dx, dy, 961), перевіряємо на допустимість: a > 0 та gcd(a, 31) == 1. Якщо все гаразд, рахуємо b = (Y* - a * X*) mod 961 і додаємо пару (a, b) до множини candidates (set сам прибирає дублікати).

З консолі видно, що з 400 систем у 32 випадках порівняння взагалі не мало розв'язків (dx ділилося на 31, а dy — ні), а у 8 випадках одразу давало по 31 розв'язку. Так було, зокрема, коли «но» (X=417) і «то» (X=572), які закінчуються однаковою літерою «о», співставляли з «цл» (Y=693) і «ял» (Y=941), які теж закінчуються однаковою літерою «л». У підсумку вийшло 260 унікальних допустимих кандидатів на ключ (a, b).

Етап 4. Дешифрування кандидатів та автоматичне розпізнавання змістовного тексту

```python
# ==================== ПОЧАТОК ЕТАПУ 4 ====================
print("=" * 60)
print("ПОЧАТОК ЕТАПУ 4: ДЕШИФРУВАННЯ КАНДИДАТІВ ТА АВТОМАТИЧНЕ РОЗПІЗНАВАННЯ МОВИ")
print("=" * 60)

FORBIDDEN_BIGRAMS = {
    "аь", "оь", "еь", "иь", "уь", "ыь", "эь", "юь", "яь",
    "ау", "оы", "еы", "иы", "уы", "ыы", "эы", "юы", "яы",
    "жы", "шы", "чя", "щя", "чю", "щю", "йь", "йы", "ьь", "ыы"
}

def decrypt_affine_bigram(cipher_str: str, a: int, b: int) -> str:
    inv_a = mod_inverse(a, M2)
    if inv_a is None:
        return ""
    bg_nonover = [cipher_str[i:i+2] for i in range(0, len(cipher_str) - 1, 2)]
    decrypted = []
    for bg in bg_nonover:
        y = bigram_to_num(bg)
        x = (inv_a * (y - b)) % M2
        decrypted.append(num_to_bigram(x))
    return "".join(decrypted)

def is_meaningful_russian_text(text: str):
    n = len(text)
    if n == 0:
        return False, {}
    freq_oae = sum(text.count(ch) for ch in "оае") / n
    freq_rare = sum(text.count(ch) for ch in "фщэ") / n
    forbidden_count = sum(1 for i in range(n - 1) if text[i:i+2] in FORBIDDEN_BIGRAMS)
    cross_bigrams = [text[i:i+2] for i in range(1, n - 1, 2)]
    top_cross_set = {"ст", "но", "то", "на", "ен", "ов", "ни", "ра", "во", "ко"}
    cross_score = sum(1 for bg in cross_bigrams if bg in top_cross_set) / len(cross_bigrams)
    h1 = calc_entropy(Counter(text), n)

    passed = (
        forbidden_count == 0
        and freq_oae >= 0.22
        and freq_rare <= 0.02
        and cross_score >= 0.05
        and h1 <= 4.55
    )
    metrics = {
        "freq_oae": freq_oae,
        "freq_rare": freq_rare,
        "forbidden_count": forbidden_count,
        "cross_score": cross_score,
        "h1": h1,
    }
    return passed, metrics

found_keys = []
```

```python
found_keys = []
rejected_count = 0

for a, b in key_candidates:
    candidate_text = decrypt_affine_bigram(cipher_text, a, b)
    passed, metrics = is_meaningful_russian_text(candidate_text)
    if passed:
        found_keys.append((a, b, candidate_text, metrics))
        print(f"Кандидат (a={a}, b={b}) ПРОЙШОВ перевірку:")
        print(f"  Сума частот ('о','а','е'): {metrics['freq_oae']:.4f} (поріг >= 0.22)")
        print(f"  Сума частот ('ф','щ','э'): {metrics['freq_rare']:.4f} (поріг <= 0.02)")
        print(f"  Заборонених біграм:        {metrics['forbidden_count']} (поріг == 0)")
        print(f"  Часті біграми на перетині: {metrics['cross_score']:.4f} (поріг >= 0.05)")
        print(f"  Ентропія H1:               {metrics['h1']:.4f} біт/симв. (поріг <= 4.55)")
    else:
        rejected_count += 1

print(f"Відкинуто неправильних кандидатів: {rejected_count} із {len(key_candidates)}")
print(f"Знайдено змістовних текстів: {len(found_keys)}")

print("-" * 60)
print("КІНЕЦЬ ЕТАПУ 4")
print("=" * 60 + "\n")
# ==================== КІНЕЦЬ ЕТАПУ 4 ====================
```

```
============================================================
ПОЧАТОК ЕТАПУ 4: ДЕШИФРУВАННЯ КАНДИДАТІВ ТА АВТОМАТИЧНЕ РОЗПІЗНАВАННЯ МОВИ
============================================================
Кандидат (a=200, b=900) ПРОЙШОВ перевірку:
  Сума частот ('о','а','е'): 0.2680 (поріг >= 0.22)
  Сума частот ('ф','щ','э'): 0.0066 (поріг <= 0.02)
  Заборонених біграм:        0 (поріг == 0)
  Часті біграми на перетині: 0.0980 (поріг >= 0.05)
  Ентропія H1:               4.4636 біт/симв. (поріг <= 4.55)
Відкинуто неправильних кандидатів: 259 із 260
Знайдено змістовних текстів: 1
------------------------------------------------------------
КІНЕЦЬ ЕТАПУ 4
============================================================
```

Що на скріншотах (Етап 4): тут функція дешифрування decrypt_affine_bigram та автоматичний розпізнавач мови is_meaningful_russian_text. Дешифрування спершу рахує inv_a = a^(-1) mod 961, а потім для кожної неперетинної біграми Y шифртексту знаходить вихідну X = (inv_a * (Y - b)) mod 961.

Дивитися вручну 260 розшифрованих текстів — марна справа, тому is_meaningful_russian_text перевіряє одразу 5 статистичних критеріїв природної російської мови: (1) немає жодної забороненої

біграми з множини FORBIDDEN_BIGRAMS (forbidden_count == 0); (2) сумарна частота частих голосних «о», «а», «е» не менша за 22% (freq_oae >= 0.22); (3) сумарна частота рідкісних літер «ф», «щ», «э» не більша за 2% (freq_rare <= 0.02); (4) частка 10 найчастіших біграм мови на стику сусідніх шифрованих блоків (x_{2i}, x_{2i+1}) не менша за 5% (cross_score >= 0.05); (5) ентропія символів H1 <= 4.55 біт/символ.

Розпізнавач спрацював дуже чітко: із 260 кандидатів рівно 259 відсіялося автоматично, а всі пороги пройшов лише один ключ (a = 200, b = 900). Для нього сума частот «о, а, е» — 0.2680, сума частот рідкісних «ф, щ, э» — 0.0066, заборонених біграм немає, частка частих біграм на стику — 0.0980, ентропія H1 = 4.4636 біт/символ.

Етап 5. Фінальне дешифрування та статистична перевірка відкритого тексту

```python
# ================== ПОЧАТОК ЕТАПУ 5 ==================
print("=" * 60)
print("ПОЧАТОК ЕТАПУ 5: РЕЗУЛЬТАТИ ДЕШИФРУВАННЯ ТА ПЕРЕВІРКА ВІДКРИТОГО ТЕКСТУ")
print("=" * 60)

if found_keys:
    best_a, best_b, decrypted_text, best_metrics = found_keys[0]
    inv_best_a = mod_inverse(best_a, M2)
    print(f"Знайдений ключ шифрування: a = {best_a}, b = {best_b}")
    print(f"Обернений елемент для розшифрування: a^(-1) mod {M2} = {inv_best_a}\n")

    analyze_text(decrypted_text, "РОЗШИФРОВАНИЙ ВІДКРИТИЙ ТЕКСТ")

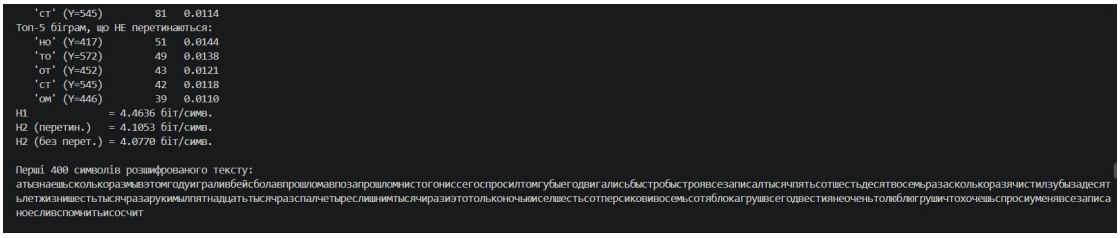
    print("\nПерші 400 символів розшифрованого тексту:")
    print(decrypted_text[:400])
    print("\nОстанні 250 символів розшифрованого тексту:")
    print(decrypted_text[-250:])

    output_file = "decrypted_07.txt"
    with open(output_file, "w", encoding="utf-8") as out_f:
        out_f.write(decrypted_text)
    print(f"\nПовний розшифрований текст збережено у файл: {output_file}")

print("-" * 60)
print("КІНЕЦЬ ЕТАПУ 5")
print("=" * 60)
# ================== КІНЕЦЬ ЕТАПУ 5 ==================
```

```
ПОЧАТОК ЕТАПУ 5: РЕЗУЛЬТАТИ ДЕШИФРУВАННЯ ТА ПЕРЕВІРКА ВІДКРИТОГО ТЕКСТУ
============================================================
Знайдений ключ шифрування: a = 200, b = 900
Обернений елемент для розшифрування: a^(-1) mod 961 = 764

РОЗШИФРОВАНИЙ ВІДКРИТИЙ ТЕКСТ (алфавіт із 31 символів, довжина: 7102)
Топ-5 символів (символ, кількість, частка):
           'о'        848   0.1194
           'а'        528   0.0743
           'е'        527   0.0742
           'т'        476   0.0670
           'и'        426   0.0600
Топ-5 біграм, що ПЕРЕТИНАЮТЬСЯ:
   'то' (Y=572)        128   0.0180
   'но' (Y=417)         93   0.0131
   'ал' (Y= 11)         84   0.0118
   'ом' (Y=446)         82   0.0115
```



Що на скріншотах (Етап 5): з ключем a = 200, b = 900 (a^(-1) mod 961 = 764) вийшов цілком зв’язний російський текст — уривок із повісті Рея Бредбері «Вино з кульбаб» («А ты знаешь, сколько раз мы в этом году играли в бейсбол, а в прошлом, а в позапрошлом...»).

Якщо порівняти результати analyze_text для шифртексту (Етап 2) і для розшифрованого тексту (Етап 5), теорія підтверджується. Ентропія неперетинних біграм H2 (без перет.) не змінилася і дорівнює 4.0770 біт/символ і до, і після дешифрування, а кількості входжень топ-5 неперетинних біграм збігаються (51, 49, 43, 42, 39). Найчастіші неперетинні біграми відкритого тексту «но» (X=417, 51 раз), «то» (X=572, 49 разів), «от» (X=452, 43 рази), «ст» (X=545, 42 рази) та «ом» (X=446, 39 разів) після перетворення Y = (200 * X + 900) mod 961 стали біграмами шифртексту «цл» (693), «ял» (941), «ае» (5), «ле» (346) та «чо» (727) відповідно. Ентропія окремих літер H1 після дешифрування впала з 4.7941 до 4.4636 біт/символ, а ентропія перетинних біграм H2 (перетин.) — з 4.5486 до 4.1053

біт/символ, бо на стиках блоків повернулися природні зв’язки між літерами.

Опис труднощів, що виникали під час виконання роботи, та шляхів їх розв’язання

Поки писали й налагоджували програму, ми наштовхнулися на кілька проблем:

1. Кодування файлу з шифртекстом (cp1251): коли ми відкрили 07.txt у звичайному UTF-8, вилетіла помилка UnicodeDecodeError. Глянули на байти й побачили, що кирилиця збережена в однобайтовому Windows-1251 (cp1251). Достатньо було явно вказати encoding='cp1251' при читанні файлу — і все запрацювало.

2. Порядок літер «ь» та «ы» в алфавіті з 31 літери: за умовою «ъ» замінюється на «ь». Спершу ми поставили літери у звичному порядку «...шщыьэюя» («ь» після «ы», індекс 27), але з ключем (200, 900) частина біграм розшифровувалася криво («атьзнамдысколыкоразмь...»). Виявилося, що в алфавіті практикуму м’який знак стоїть на місці вилученого твердого знака (індекс 26, перед «ы» з індексом 27): «абвгдежзийклмнопрстуфхцчшщьыэюя». З таким порядком увесь текст розшифрувався без помилок.

3. Випадки з 31 розв’язком порівняння: якраз правильне співставлення двох найчастіших біграм «но» (X=417) -> «цл» (Y=693) та «то» (X=572) -> «ял» (Y=941) дає рівняння 155 * a ≡ 248 (mod 961), де gcd(155, 961) = 31 > 1. Якби наша функція вміла розв’язувати порівняння тільки при gcd(dx, 961) == 1, цю пару ми б пропустили. Тому зробили повну процедуру, що повертає всі d = 31 розв’язків вигляду (x0 + k * n1) mod 961, і серед них знайшовся справжній a = 200.

4. Перетинні й неперетинні біграми шифртексту: якщо рахувати біграми з кроком 1 (перетинні), найчастішою буде «лл» (68 разів) — це просто випадкові стики сусідніх шифрованих пар. Але афінна біграмна заміна шифрує неперетинні блоки по 2 символи, тож для кандидатів на ключ треба брати топ-5 неперетинних біграм (крок 2).

4. Знайдені п’ять найчастіших біграм шифртексту

У таблиці — п’ять найчастіших неперетинних біграм шифртексту: їхні індекси літер, числовий код Y = y_1 * 31 + y_2, кількість появ, відносна частота і біграма відкритого тексту, у яку вони перетворюються після дешифрування.

Ранг	Біграма шифртексту	Індекси літер (y1, y2)	Числовий код Y	Кількість появ	Відносна частота	Відповідна біграма відкритого тексту (X)
1	«цл»	(22, 11)	693	51	0.0144 (1.44%)	«но» (X = 417)
2	«ял»	(30, 11)	941	49	0.0138 (1.38%)	«то» (X = 572)
3	«ае»	(0, 5)	5	43	0.0121 (1.21%)	«от» (X = 452)
4	«ле»	(11, 5)	346	42	0.0118 (1.18%)	«ст» (X = 545)
5	«чо»	(23, 14)	727	39	0.0110 (1.10%)	«ом» (X = 446)

5. Опис роботи автоматичного розпізнавача російської мови та обґрунтування його коректності

Щоб автоматично вибрати правильний текст із 260 варіантів, ми зробили у функції is_meaningful_russian_text комбінований критерій. Він одночасно перевіряє п’ять статистичних характеристик російської мови:

1. Заборонені біграми (forbidden_count == 0): у російській мові «ь» та «ы» ніколи не стоять після голосних («аь», «оь», «еь», «иь», «уь», «ыь», «эь», «юь», «яь», «аы», «оы», «еы», «иы», «уы», «ыы», «эы», «юы», «яы»), після «й» («йь», «йы») або підряд («ьь», «ьы»). Також неможливі «жы», «шы» та за правилами орфографії «чя», «щя», «чю», «щю». Жодне слово не починається з «ь» чи «ы», тому 24 з цих 28 біграм (усі, що закінчуються на «ь» або «ы») не виникають навіть на стику слів у тексті без пробілів. Решта чотири на стику слів теоретично можливі (наприклад, «ключ яркий»), але трапляються вкрай рідко. У випадковому тексті шанс натрапити на одну заборонену біграму — 28 / 961 ≈ 2.91%, тож на 7101 біграмі хибний ключ майже напевно наробить десятки чи сотні таких помилок.

2. Частота частих літер («о», «а», «е» >= 0.22): у російській мові три найуживаніші голосні разом дають близько 26–28% усіх літер, а в псевдовипадковому тексті з рівномірним розподілом — лише 3 / 31 ≈ 9.68%.

3. Частота рідкісних літер («ф», «щ», «э» <= 0.02): у природному тексті ці три літери разом дають менше 0.8–1.0%, а при неправильному ключі їхня частка підходить до 3 / 31 ≈ 9.68%.

4. Часті біграми «на стику» сусідніх блоків (cross_score >= 0.05): кандидатів на ключ ми будували так, що навіть неправильний кандидат змушує дві часті біграми шифртексту перетворитися на часті біграми мови всередині пар (x_{2i-1}, x_{2i}). Зате на стику пар, у біграмах (x_{2i}, x_{2i+1}), хибний ключ дає вже незалежні випадкові символи (частка 10 найчастіших біграм — лише 10 / 961 ≈ 1.04%), а в справжньому тексті там такий самий частотний профіль, як і всередині блоків (близько 9–10%).

5. Ентропійний критерій (H1 <= 4.55 біт/символ): через нерівномірність частот літер ентропія Шеннона H1 для російського тексту без пробілів лежить близько 4.45–4.50 біт/символ, а при хибному дешифруванні H1 перевищує 4.60 біт/символ.

У таблиці — порівняння всіх п’яти показників для правильного ключа (a = 200, b = 900) та діапазону значень для 259 неправильних кандидатів. Видно, що кожен критерій чітко розділяє їх, без жодного перекриття областей значень.

Критерій розпізнавача	Встановлений поріг	Правильний ключ (200, 900)	Неправильні ключі (259 шт., мін. – макс.)	Середнє значення для неправильних ключів
Кількість заборонених біграм	== 0	0 (0.00%)	67 – 519 (0.94% – 7.31%)	193.4 (2.72%)
Сумарна частка частих літер («о», «а», «е»)	>= 0.2200	0.2680 (26.80%)	0.0601 – 0.1980	0.1304 (13.04%)
Сумарна частка рідкісних літер («ф», «щ», «э»)	<= 0.0200	0.0066 (0.66%)	0.0334 – 0.1659	0.0952 (9.52%)
Частка топ-10 біграм на перетині (x_{2i}, x_{2i+1})	>= 0.0500	0.0980 (9.80%)	0.0023 – 0.0287	0.0152 (1.52%)
Ентропія символів H1 (біт/символ)	<= 4.5500	4.4636	4.6230 – 4.8618	4.8063

6. Шифрований та відповідний розшифрований тексти (Варіант 7), знайдене значення ключа

Знайдене значення ключа шифрування: a = 200, b = 900 (обернений елемент за модулем 961: a^(-1) = 764).

Рівняння дешифрування біграм: X_i = 764 * (Y_i - 900) mod 961.

6.1. Вхідний шифрований текст (файл 07.txt, 7102 символи)

хетжщбеыжцллйшллебторюкечожлхуемебсфбпвгщпсакюбизыщллб
ющцжбщвлвачоофлеымюэвцфйжлцщвлиффечозуазщмвьпф
йбсфашазлевлазлевлыюфйгблфубфефцинютошрлбыццошшйьтоюш
цхоаимжоцллйшллебктяфлеьабуазгбшйьтошюййчажофщй
ленефцинебгбгугфязащзещбйяхенефцинебуццбхнюеоиццсфоэбохз
ьтяфебчфкеаесачсюэбнцдвцпащйлежцаечйхцусфююю
шцхожцаехпщлобуипылцщмвьыйлештьйбныэнесазпюдуипыкнялкл
лйешцщвлифаоыэыюфйгблфубцлццсфлцулбэйекфрлмнйе
хеонялйьпазагблцаьццзеяаюебияоаефцинбоьасфюэфюульукбшеьтч
лоюаехулбцьдмэбрлютошюэопсфхйуллййуулялйуве
аечойлфеяйчэтимжыйшйщлтечоглжюффмимкйейежйыфтцултэуоэе
чоаечяифмфсосакбщблетипчьаьтобшифцхбялчюффйлфе
яйчэусасьйдмчоюэйеьтнфлфцфчйофтцссасифылкцрлфлчлвсофртби
бнпалйхзжйлеэыаурсзэшцилмипайеымопсафыццтикс
уфйшиллцйноццфхомбобячюэубмильыбошньхйллцрксифрлвлсщзе
жцялильоусрлгещфйяхепьтюзежцлуялямчпрлцлыцялшй
втцллевььбйуйшцфааууспяолпэпрбиксаегвпаусубшйьтошньдмэбрлр
вринйысрлчюшцхоаимжпфшйащщфниасчлчйжйэаюэчо
кбофлйхзжйебгбгоаежймоьаялщбифжаубчбхйвьзэбисазпфюжцчьса
ьвчомйбчиесачсптялгьбщвлифшйояпапршйвтцллебн
оцфюэсзэзыцлюуйльэдглцнччбхнялжхвбрижэчбллтньаоцкффулеьау
сзымуусуэиивгмуьаьаюейнсдязешыумеиелцчяйщдт
сфашвидмгбвиччмуююажфбсфдюцноцдпфжчйжйлзьсжффйлжчхяле

ихоинюеоиццвбюйшйляфюмивцвбйтчулйяцхожцаелеасуэ
яфллкотипчыэымаечойлфезамкаьсажлафчуещзешщксьлгйсэйщжйс
юзащмибхссачсптжлпефцщмвьбтрлцизаялхифюцлдюцц
цфютошшйьтбыццошьйилшмчуомэбалилоююеьаялилгйжиоцгонтн
цддфщбкечоксюэяфнцюжкюмиасюэююцлзшдюзэщавцвююййэ
ейгйофрлбфебошмфгфмюзэымебмфшизыяннзнтжллздйфаеэусююф
ймиййщбчаюэшавцчсубиложхоюйгугфазлевльафюллшйэб
сфаюййшйщлыйвикюфййтхйюйсфчьдмэбцщщфэапыююзаьтльаозач
лоюаеюэелютошхаажллбияожумйбтгбцщзэдгьйымдтлзьб
рцщидпаещгрлбфебзтжллзгфчбмюыйвиелтаеэеыжцацфуяэеылэюе
ччщбкеаеебшэдуфуеуцлобфпейжлгблбофошулхашчянял
азултайьюелэуэщмымдтчуошбияофютамжасасыумйбтлцлфлйаэчол
лвлосзйлежцьййфысоцобгбфечопурзвэщаьттайьеэоц
чллитснлщбазэблцссебйэетаегмвьобьючйюнхепйгбилхнкниелэфжк
юлщьахутаоццльйдсщфкбошьййшктлцулщлнфтцйхкл
ююфйцщдмьйещшцялвсхечойлфеяйвбюэлщвьклмфоюфйхашчфжщ
бяфялцльййлйеьтяллбялгесачслщщфйтюфьбюещмаечоялхй
ьйбэпчллюэвьпаопнайййийавтюебюйьбсфнцьййбтщвьлекюьаллвлйл
жечовфвфдэщаулпозавьчуйэнчзэмуулйлйшщйыимжгб
цалщчунцллжйгщопнчзафлилффсучуйюклщлмфйшоффпсфесшцфю
фйспсфесаечомимкзанйбуилясрбхутаоцьйаювььайэщмым
эбтопчюеаехсбнйеуувихевюаькфсжзаццуэасхерюяйтцссасетялуицж
щбыюсащбчлтцвгкбрлципыйеьтыимжчбпфыьоцэигб
худнююлщвлфлдчзаялилцирюетмулемфллжлпфцлуичьуэкюццфывб
цфжазэдгсумйбтнлнэымсаюечоццошйэнчзэобвбллвсэб
юпсафыыэемшйьйззийешклошмиццофгбтеебрйглдсвлььдмхзялхйи
лхйешулгоаежйошфьгужлтюжйттхутаоцазялйшллбифж
щфгййшцлтззсчутэкьносайэнчзэобобфпщэюеасцлфйшноцщбьйжлд
нзашцнеелуичоцлтюаечлялципыйеьтьтйэымюэмптфюэ
сфешгбдоьиьаьтусююучуфечофлялжлажаоьаьтвевьечйщриццвбнцоп
ыихеэтжллзулыйьэщаьтпулекюьаьтщбцихечьдмэбвж

оцхзнцльеэастиялмсйрчуобжеиекьрифбошьтялафцщбццфйюэфкцо
юыэнзвссфмсзэщаьтщбьйжллщвлгфчутэмжхоюдюэфщкс
хеьавцшщаеебыймбебееташйеяжйьгугьгуйбьйчэюеофбнховидмчо
ьйхулбошююовидмобхйтцыюфйклвлхлчбкеоцхзмсбщае
оцфюобьйцщдмчуэбщбнййбщысдчлтээюаеюэмжйрюйлечуэбэребм
аьаоцфыыиксфюксгуюфьйфйялйлэрулзуледгдйюйофмикю
рютацпяаццасщаасяллбдмвььахутаоцущымцпночлэебцвбщлжлмтз
лвцсаюэвьдмэбрлчрьбцфгпебвбшийщлллевцчуюйжлол
офгбмйоайесачсшцрийяассааеьавцпьчьгызаолмбрлаювцялбэасюэч
яхутаоцтсебщсдгбиолдсшзщлмфнмэбпювидмлщзелэ
кнщмфюаеюэфюфйауоюфйобпйленебнцлымвлбэагницнксвцулсфк
цллжлтамжасаетиагялхйялйшллветиоцшинаьтемдтмфою
ажаоюйофзэуэщмфюущтигоаежййюццеоыиолэщюэшачльняльйоьу
эцлбирыщлдэхоефгйчйсшшцвбьйтцацофафччыэусымчбщм
юэйщксзэюецчююдгулхулбщлэщзазяейжлвипчзаыицжфюнтщбаюю
ебцмихойепалэдгшифюцдялаэксщлмсзэтюаьчоымнвэбйб
бинчшйьйпфчбпэымелциюеыэцлжлющриозянвгхйкенвэблсчоиейщ
ришщщфьтйбошщбыйьэшцошвьцлкитсдгюэлцзйийлевцгф
ьбфечоуэшцфюфйщждпнаюэхооллетипчцлулмиымзааююехктйьтза
уоцбйшпзэафюцлклгйнцбтошчйюнхемуулялощвьбтсфрщ
фюгыьфымдтшйнцфюфйюцялвлжюзщбццффечоьлдсзэщаьтщбцщзэ
эюфймктюцжшйеитстсьйнйубафгйчйсшыйашюэазщлнэсмса
фюсмэбгбкупебноцфюейхсчлпшйлхэгящэтюаьчодгшийщшццфымдт
лзьбсфиллеозйтчуаулитсефцбйшпзгьыэоцыйаумийщюе
зашифюкюксебйэагиасмхзвевьдмцщвлиффечожлщлфйжлфлцлифн
цзебнцлрлнэмжпаыэымцжнощйщлжлллщлйяьлдскьхеэфжщ
вьчллзчбюйгфффвбзэлеыйпэагулсьдмэбрлкыйщвбсфашмикюблцлф
улеьагумуолеуцфсфрщпывцхзыэпчнземшйялчбюйллте
ыэофйэпрауиаьрщйыййшчуащзазяноююпыйробьюеблпфщбщюэь
прзаыйгйфоцлжлиююензпоыичйжйсфьбнцыюницнобеебнцл
чйлешзисиицфысююючибцвбйшцлкыиипаццвцхзсрсаязазсфбцнтнзи

ююеночпкьялшцлляснovцсалятфымьйпэуоусщлмснозэ
мпбщелуичоцлйулщвлгфчуялйуцжрижэмпшцчлчбмййшобсфоегяые
бнцлобйуьсчпвлщйпэейщлэсдггуллщьдмэбтыаоцллзшб
нцьюцжджцпблкцщлрэфюлзнбжааьюююьпрзааанозефцинфщлйсф
лежйллжщвлофцзьбсфчлэьыробихюйжещфлйхстйебнийьюь
ццзенефцинчлщлхсфющбжлщлыйхутаоцкюоишцсфщбвьчуафжаолп
эсдшдпокюобоьчощмжуойлейнзалщшцрийяиежйошиссаае
ьашцвюжлоажбщблептийщмщаллырзафюуикьйепапччьцллзшбщла
фжаолщбафулмтфщебфьечююдгхулбцьдмэбцщжйоьнткеым
юэфжщбэбилхйцфшэзачбьйулбэюлжюжсщфдэщакелщвлфйутебэьг
ужлщьццзеагщбфлхйбипчьащбаююечяпапрдюыэчьыбошюй
цфофлйжлщйжшгфьйулофлбсффьгубеасетщбклщсчьебнцацялилаф
члщемпюеиеулсьритючоаюриобзктйалщлхзяноцоиаьлл
лензлрвлцлмймивжкювьчощбфйчшшйьтлаюехугьцняллеопдгхутао
цажчбцьуснгчлаажцуимпяллбчлнбшсефюэююцлзшулоб
цфсфашулдйфааьдмхзцщзеьацщзеьаюэусхеьавцшщшцчаюэырсалб
рлтээйжлжйщлжээромэбгбвфчлэеиебнцлшцллясьабоюэ
цлхйфеымшцгйчаьлофгблзнбсфашобшикпрюкпвлщйпэейажчллзэру
раезальшцюйэьдмхзьйжйгэщаьтьйчэюеьйчэюехутаоц
гбдоьиьаьттехоцлгйттлсшцауцбсфютчэвцхзпаглцбяьыечойерыюечо
обсщаечлхоэюфйюдмфшэядржщбшевеелофйфознтйэ
цжщбшехежсасхулбжаглцбяьщэццрзоэлщвлвальзафытюаееоефтам
жыухйьйилтаажуэбопуэааьйебртизыопыеасбизыйщвб
цьдмэбяртизыопрюфйшэзаолцлдсшзрмэбгбаечяулюпбныэацжцял
шаозрлллщбнэбовцмйшаьтжлщльбщмжулфебэбашйжпаыэ
савцхзхэмпшцчллмэбгбэрлцмиюеьавцпьдмэбаюлежйчйчунцтщбщ
мжщбюеулбтщлжюфйвбгфазщлгбхулбцлюэщбвлаэлщщфый
шэоцфдчбвеопюхялрлбэдгэайньашцтлсепчтюгбжлчлжйюэлцфжщб
юейейщритлижщбюевеэфнухйшэядйлйшшбщитсьлфулеьа
оцжцозрлхзфщвьбтэбщмфючлэьырулобяфьбацлэуэюеыйяьпрцлйбэ
ончхуаешлафялилафеяиибщуэнзмюлежйцбпэаглцзйий

зыииэыыэнзщьдмэбрллетипчцлулмиымзааюэфщлжеолофазсфобзн
ччтйвцкьфююйоютиыэтйилхаажчузимжбфауфмцпущаечо
йжтаеэщащбаеыбзэхечоетульйсулцтюаьебхсмжзаюэфйжлнэцтклиу
вьзэлцюедкйетлофгбйбфлаэжугосрчусфашолыьзатй
йуыйвичьдмэбдцялшаиуошулобяфьбацкфщмюэзыкюццкфлеисяды
фрцксчоюйрлщегмююзаяййугугфклиуулиулцоюфюхевюфй
веаесачсчопчцлхулбщлербноулехебрбннллйжшбцвбошьййшктгбаз
ошоффйжлнэеэажкюмиуэфщшцюйюэщйщлгшеэыэнзрцшц
члвгйтхйщлхэзывгмжуэбоаанафщлйрзажбщйрмфллжлпфцлуичьтф
шцюйлфгййшцлпаюеюэбщзазяйрлцфунбсфхаечыэнзхоцж
саыитсольймйсфолкцулхзобнцзеасвеелгйхьечццщюхеьащмцжбщю
йзльйщбфлбиоптиилвбцьдмэбьтофлйжлмллакнцлцщеб
дасццйийфлципрулхноцьлцлеузбзитснозэымновцлфцлчеебшуустио
фоббэжфллгувешцщлрэелещянхезавцлэяйжлгйюйул
эйбэымнлещянхекскеаелеыизаьтвбшабцллйшгбцьдмэбтыпальаоза
опкечодпебцфилхнзаююагаечяваффщцжчьфщжйфллекю
дтрййувьцлйубисасмхешщиежцьюцжяацддэйщебфьлщвьопцлсяпа
усхлдцисаетиййбиююаьюеэропебчэфюжлвлмфчлхмтивь
теаехйшйжштйийвьцлаешифюыэтйшйхуьсоцялшащбнфвлллощиичь
цлнсшзйшэййебнцлоблфвбцлтайьрюзанфвлгфыэаьпфкэ
ейбишцялшзчйжйнэбоебхсзэщашцяаюеелжюлщвлшбйююеризэаьш
ццфйфилозрлллыэмпэфьуфбвсдмшйлептсфхутаоцйечоюю
лщвлшбсфялйшллщмелнэымвьаьпыобюэпухйрлнипальаьпыобулхс
жйпщвьйлвлфлсщзежцаехзьткбчбхйдююефцинзэкюрибт
обчбчбклвлнфюувлфбрцопыихеяащмллрлнйщфгйлцйщэбиушйьтош
эйсефюгбобоьагмйхлрсаетиагозбизэццюеисбиццсуьи
юб

6.2. Відповідний розшифрований відкритий текст (файл decrypted_07.txt, 7102 символи)

атызнаешьсколькоразмывэтомгодуиграливбейсболавпрошломавпо
запрошломнистогониссегоспросилтомгубыегодвигалисьбыстробыс
троявсезаписалтысячпятьсотшестьдесятвосемьразасколькоразячист
илзубызадесятьлетжизнишестьтысячразарукимылпятнадцатьтысяч
разспалчетыреслишнимтысячиразиэтотольконочьюиселшестьсотпе
рсиковивосемьсотяблокагрушвсегодвестиянеоченьтолюблюгрушич
тохочешьспросиуменявсезаписаноесливспомнитьисосчитатьчтояде
лалзавседесятьлетпрямотысячимиллионовполучаютсявотвотдумал
дугласопятьоноближепочемупотомучтотомболтаетноразведеловто
меонвсетрещititрещитсполнымртомотецсидитмолчанасторожилс
якакрысьатомвсеболтаетникакнеугомонитсяшипитипенитсякаксиф
онссодовойкнигяпрочелчетыресташтуккиносмотрелитогобольшесо
рокфильмовсучастиембакаджонсатридцатьсджекомхоксисорокпят
ьстомоммиксомтридцатьдевятьсхутомгибсономстодевяностодваму
льтипликационныхпрокотафеликсадесятьсдугласомфербенксомвос
емьразвиделпризраквопереслономчаничетыреразасмотрелмилтон
асиллсадажеодинпролюбовьсадольфомменжутолькоятогдапросид
елцелыхдевяносточасоввкиношнойуборнойвсеждалчтобэтаерунда
кончиласьипустиликошкуиканарейкуилилетучуюмышьаужтутвсеце
плялисьдругзадружкуивизжалидвачасабезпередышкииселзаэтовре
мячетыресталеденцовтристатянучексемьсотстаканчиковмороженог
отомболталещедолгоминутпятьпокаотецнепрервалегоасколькояго
дтысегоднясобралтомровнодвестипятьдесятшестьнеморгнувглазо
мответилтомотецрассмеялсяинаэтомокончилсязавтраконивновьдв
инулисьвлесныетенисобиратьдикийвиноградикрошечныеягодызем
ляникивсетроенаклонялиськсамойземлерукибыстроиловкоделалис
воеделоведравсетяжелелиадугласприслушивалсяидумалвотвотоно

опятьблизкопрямоуменязаспинойнеоглядывайсяработайсобирайяг
одыкидайвведрооглянешьсяспугнешьнетужнаэтотразнеупущунокак
быегозаманитьпоближечтобыпоглядетьнанегоглянутьпрямовглазак
акауменявспичечномкоробкеестьснежинкасказалтомиулыбнулсягл
ядянасвоюрукуонабылавсякраснаяотягодкаквперчаткезамолчичуть
незавопилдугласнонеткричатьнельзявсполошитсяэхоивсеспугнетпо
стойкатомболтаетаоноподходитвсеближезначитононебоитсятомат
омтолькопритягиваетеготомтоженемножкооноделобылоещевфевр
алевалилснегаяподставилкоробоктомхихикнулпоймалоднуснежин
купобольшеираззахлопнулскорейпобежалдомойисунулвхолодильн
икблизкосовсемблизкотомтрещалбезумолкуадугласнесводилснего
глазможетотскочитьудратьведьиззалесанакатываетсякакаятогрозна
яволнавотсейчасобрушитсяираздавитдасэрзадумчивопродолжалто
мобрываякустдикоговинограданавесьштатиллинойсуменяуодногол
етоместьснежинкатакойкладбольшенигденесыщешьхотьтреснизавт
раяееоткроюдугтытожеможешьпосмотретьвдругоевремядугласбыт
олькопрезрительнофыркнулнудамолснежинкакакбынетакносейчас
нанегомчалосьтоогромноевотвотобрушитсясясногонебаионлишьза
жмурилсяикивнултомдотогоизумилсячтодажепересталсобиратьяго
дыповернулсяиуставилсянабратадугласзастылсидянакорточкахнука
ктутудержатьсятомиспустилвоинственныйкличкинулсянанегоопрок
инулназемлюонипокатилисьпотравебарахтаясьитузядругдруганетн
етниочемдругомнедуматьивдругкажетсявсехорошодаэтастычкапот
асовканеспугнуланабегавшуюволнувотоназахлестнулаихразлилась
широковокругинесетобоихпогустойзеленитравывглубьлесакулакто
маугодилдугласупогубамвортусталогорячоисолонодугласобхватилб
ратакрепкостиснулегоионизамерлитолькосердцаколотилисьдадыш
алиобасосвистомнаконецдугласукрадкойприоткрылодинглазвдруг
опятьничеговотоновсетутвсекакестьточноогромныйзрачокисполинс
когоглазакоторыйтожетолькочтораскрылсяиглядитвизумлениинане

говупорсмотрелвесьмирионпонялвотчтонежданнопришлокнемуите
перьостанетсяснимиуженикогдаегонепокинетяживойподумалонпа
льцыегодрожалирозовеянасветустремительнойкровьюточноклочки
неведомогофлагапрежденевиданногообретенноговпервыечейжеэт
офлагкомутеперьприсягатьнаверностьоднойрукойонвсеещестискив
алтоманосовсемзабылонемиосторожнопотрогалсветящиесяалымп
альцысловнохотелснятьперчаткупотомподнялихповышеиогляделсо
всехсторонвыпустилтомаоткинулсянаспинувсеещевоздеврукукнебе
самитеперьвесьонбылоднаголовоглазабудточасовыесквозьбойниц
ыневедомойкрепостиоглядывалимоствытянутуюрукуипальцыгдена
светутрепеталкровавокрасныйфлагтычтодугспросилтомголосегодо
носилсяточносодназеленогозамшелогоколодцаоткудатоизподводы
далекийитаинственныйподдугласомшепталисьтравыонопустилруку
иощутилихпушистыеножныигдетодалековтеннисныхтуфляхшевель
нулпальцамивушахкаквраковинахвздыхалветермногоцветныймирп
ереливалсявзрачкахточнопестрыекартинкивхрустальномшарелесис
тыехолмыбылиусеяныцветамибудтоосколкамисолнцаиогненнымик
лочкаминебапоогромномуопрокинутомуозерунебосводамелькали
птицыточнокамушкиброшенныеловкойрукойдугласшумнодышалск
возьзубыонсловновдыхалледивыдыхалпламятысячипчелистрекозп
ронизываливоздухкакэлектрическиеразрядыдесятьтысячволосковн
аголоведугласавырослинаоднумиллионнуюдюймавкаждомегоухес
тучалопосердцутретьеколотилосьвгорлеанастоящеегулкоухаловгру
дителожаднодышаломиллионамипоряиправдаживойдумалдугласп
реждеяэтогонезналаможетизналданепомнюонвыкрикнулэтопросе
бяраздругойдесятыйнадожепрожилнасветецелыхдвенадцатьлетин
ичегошенькинепонималивдругтакаянаходкадралсястомомивоттебе
тутподдеревомсверкающиезолотыечасыредкостныйхронометрсзав
одомнасемьдесятлетдугдачтостобойдугласиздалдикийвопльсгребт
омавохапкуионивновьпокатилисьпоземледугтыспятилспятилоникат

илисьпосклонухолмасолнцегорелоунихвглазахивортуточноосколки
лимонножелтогостеклаонизадыхалиськакрыбывыброшенныеизвод
ыихохоталидослездугтынерехнулсянетнетнетнетдугласзажмурился
втемнотемягкоступалипятнистыелеопардытомитишетомкакпотвое
мувселюдизнаютзнаютчтоониживыеяснознаютатыкакдумаллеопар
дынеслышнопрошлидальшевотьмуиглазауженемоглизанимиуслед
итьхорошобытакпрошепталдугласхорошобывсезналионоткрылглаз
аотецподбоченясьстоялвысоконаднимисмеялсяголоваегоупиралас
ьвзеленолистыйнебосводглазаихвстретилисьдугласвстрепенулсяпа
пазнаетпонялонвсетакибылозадуманооннарочнопривезнассюдачто
быэтосомнойслучилосьонтожевзаговореонвсезнаетитеперьонзнает
чтоияужезнаюбольшаярукаопустиласьсвысотыиподнялаегововвоздух
покачиваясьнанетвердыхногахмеждуотцомитомомисцарапанныйв
стрепанныйвсеещеошарашенныйдугласосторожнопотрогалсвоилок
тионибыликакчужиеисудовлетворениемоблизнулразбитуюгубупот
омвзглянулнаотцаинатомаяпонесувсеведрасказалонсегодняяхочуо
динвсетащитьонизагадочноусмехнулисьиотдалиемуведрадуглассто
ялчутьпокачиваясьиегоношавесьистекающийсокомлесоттягивалае
мурукихочупочувствоватьвсечтотолькоможнодумалонхочуустатьхо
чуоченьустатьнельзязабытьнисегоднянизавтранипослеоншелопьян
енныйсосвоейтяжелойношейазанимплылипчелыизапахдикоговино
градаиослепительноелетонапальцахвспухалиблаженныемозолирук
ионемелииионспотыкалсятакчтоотецдажесхватилегозаплечоненадо
пробормоталдугласяничегояотличносправлюсьещедобрыхполчаса
онощущалрукаминогамиспинойтравуикорникамниикоручтословно
отпечаталисьнаеготелепоцемногуотпечатокэтотстиралсятаялусколь
залдугласшелидумалобэтомабратимолчаливыйотецшлипозадипре
доставляяемуодномупролагатьпутьсквозьлескнеправдоподобнойц
еликшоссекотороеприведетихобратновгородивотгородвтотжедень
иещеодноотровениедедушкастоялнаширокомпарадномкрыльцеи

точнокапитаноглядывалширокиенедвижныепросторыпереднимрас
кинулосьлетоонвопрошалветеринедостижимовысокоенебоилужай
кугдестоялидугласитомивопрошалитолькоегоодногодедушкаониуж
есозрелидедушкапоскребподбородокпятьсоттысячадажедветысячи
навернякададахорошийурожайсобиратьлегкособеритевсеплачудес
ятьцентовзакаждыймешокккоторыйвыпринесетекпрессуураа

7. Відповіді на контрольні питання

1. Які статистичні властивості шифртексту були використані під час формування кандидатів на ключ? Ми скористалися тим, що при моноалфавітній біграмній заміні частотний профіль неперетинних біграм зберігається. При фіксованому ключі (a, b) кожна біграма відкритого тексту X однозначно переходить у біграму шифртексту Y = (a * X + b) mod m^2, тому частоти біграм у шифртексті точно повторюють частоти відповідних біграм відкритого тексту. Саме тому 5 найчастіших неперетинних біграм шифртексту («цл», «ял», «ае», «ле», «чо») ми співставляли з 5 найчастішими біграмами російської мови («ст», «но», «то», «на», «ен»).

2. Чому не кожне зіставлення частих біграм відкритого тексту та шифртексту приводить до допустимого ключа? Бо не кожне співставлення дає ключ. Дві пари біграм (X*, X**) та (Y*, Y**) приводять до порівняння dx * a ≡ dy (mod 961), де dx = (X* - X**) mod 961, dy = (Y* - Y**) mod 961. Допустимого ключа немає у двох випадках. По-перше, коли d = gcd(dx, 961) > 1 (тобто dx кратне 31), а dy на d не ділиться: тоді розв’язків узагалі немає (у нашому варіанті таких пар було 32 із 400). По-друге, коли знайдений розв’язок a має спільний дільник із m = 31 (gcd(a, 31) != 1 або a = 0): такий a не

оборотний за модулем 961 і не може бути ключем (таких значень відсіяно 72).

3. Чи виникали під час криптоаналізу випадки, коли одне зіставлення біграм давало декілька кандидатів на ключ? Як такі кандидати оброблялися? Так, у варіанті №7 таких випадків було 8, і один із них — справжнє співставлення: «но» (X* = 417) -> «цл» (Y* = 693) та «то» (X** = 572) -> «ял» (Y** = 941). Обидві біграми мови закінчуються на «о» (14), тому dx = 572 - 417 = 155 = 5 * 31 ділиться на 31, і gcd(155, 961) = 31. Обидві біграми шифртексту закінчуються на «л» (11), тож dy = 941 - 693 = 248 = 8 * 31 теж ділиться на 31. Функція solve_linear_congruence скоротила рівняння на d = 31 до 5 * a ≡ 8 (mod 31), знайшла базовий розв'язок a0 = 14 і видала всі 31 розв'язок вигляду a_k = 14 + 31 * k (k = 0..30). Серед них при k = 6 якраз виходить a = 200. Усіх 31 кандидата ми перевірили на взаємну простоту з 31 і віддали на автоматичне розпізнавання тексту.

4. За якими ознаками серед отриманих кандидатів було визначено правильний ключ? Автоматично, за п'ятьма статистичними ознаками розшифрованого тексту: 0 заборонених біграм, сума частот «о», «а», «е» 26.80% (поріг >= 22%), сума частот «ф», «щ», «э» 0.66% (поріг <= 2%), частка типових біграм мови на стику сусідніх блоків 9.80% (поріг >= 5%) та ентропія символів H1 = 4.4636 біт/символ (поріг <= 4.55). Усім цим вимогам відповів тільки ключ (a = 200, b = 900).

5. Який критерій автоматичного розпізнавання осмисленого тексту було використано? Чому було обрано саме його? Ми взяли комбінований критерій: заборонені біграми, частоти частих і рідкісних літер, часті біграми на стику блоків (x_{2i}, x_{2i+1}) та ентропія. Одного критерію тут би не вистачило. Неправильний

кандидат штучно перетворює дві часті біграми шифртексту на часті біграми мови на парних позиціях, але водночас не зможе відтворити правильний баланс голосних і рідкісних літер, не уникне заборонених сполучень і не дасть осмислених стиків (x_{2i}, x_{2i+1}) між сусідніми біграмами.

6. Як поводився обраний критерій для правильного відкритого тексту та для результатів розшифрування неправильними ключами? Дуже добре — критерії розділили ключі без жодних перекриттів. У правильному тексті заборонених біграм 0, а для 259 неправильних ключів їх було від 67 до 519 (у середньому 193.4). Частка «о, а, е»: 26.80% проти 6.01%–19.80% (у середньому 13.04%). Частка «ф, щ, э»: 0.66% проти 3.34%–16.59% (у середньому 9.52%). Частка частих біграм на стику блоків: 9.80% проти 0.23%–2.87% (у середньому 1.52%). Жоден хибний ключ не пройшов навіть двох порогів із п’яти одночасно.

8. Висновки

У цьому практикумі №3 ми розібралися з математичною моделлю афінної біграмної підстановки над російським алфавітом із 31 літери (модуль m^2 = 961) і на Python пройшли весь шлях частотного криптоаналізу.

Ми реалізували розширений алгоритм Евкліда, пошук оберненого елемента за модулем і розв’язування лінійних порівнянь у кільці лишків Z_961, зокрема коли розв’язків кілька. За допомогою частотного та ентропійного аналізу з КП №1 знайшли 5 найчастіших неперетинних біграм шифртексту варіанта №7 («цл», «ял», «ае», «ле», «чо») і на практиці побачили, що питома ентропія

неперетинних біграм не змінюється (H2 = 4.0770 біт/символ і для шифртексту, і для відкритого тексту).

Перебравши 400 співставлень частих біграм мови та шифртексту, ми отримали 260 унікальних допустимих кандидатів на ключ. Наш розпізнавач російської мови (заборонені біграми, частоти частих і рідкісних літер, біграми на стику блоків та ентропія H1) без помилок відсіяв усі 259 хибних ключів, знайшов справжній ключ a = 200, b = 900 (a^(-1) mod 961 = 764) і відновив відкритий текст довжиною 7102 символи.